Trabalho Prático 1
Software Básico

1 Introdução

O trabalho consiste em implementar em C/C++ um método de tradução de uma linguagem de montagem simples para uma representação de código objeto. O tradutor a ser implementado será um Assembler da linguagem hipotética vista em sala de aula.

2 Objetivo

Fixar o funcionamento de um processo de tradução (algoritmo de 2 passagens e passagem única), etapas de análise léxica, sintática e semântica e a etapa de geração de código objeto.

3 Especificação

3.1 Montador

A linguagem de montagem utilizada será a linguagem simbólica hipotética apresentada em sala. Esta linguagem é formada por um conjunto de apenas 14 instruções. Uma diferença com o formato visto em sala de aula é que os programas devem ser divididos em seções de código e dados. Você pode escolher entre uma ou duas passagens.

Para cada instrução da máquina hipotética, deve-se usar a Tabela apresentada em sala de aula a qual contém o mnemônico, quantidade de operandos, código de operação (OPCODE) utilizado na montagem, tamanho em palavras da instrução montada. Além disso, o montador deve ser capaz de aceitar as diretivas SPACE e CONST como vista em sala de aula. SPACE pode receber UM argumento.

Os identificadores de variáveis e rótulos são limitados em 30 caracteres, simplesmente pq NUNCA vai ter um label maior que isso. Os rótulos seguem as regras comuns da linguagem C, sendo compostos por letras, números ou o caractere (underscore) e com a restrição de que o primeiro caractere não pode ser um número.

Para eliminar ambiguidade, as seções de código e dados devem ser devidamente marcadas com as diretivas correspondentes, como ilustra o exemplo abaixo. Seção de dados pode vir em qualquer antes ou depois da seção de texto no arquivo de entrada.

```
SECAO  DATA
N1: SPACE
N2: CONST -48
N4: SPACE


SECAO TEXT
ROT: INPUT N1
COPY N1 N4 ;comentario
COPY N2 N3
COPY N3 N3
OUTPUT  N3
STOP
```

O montador deve ser capaz de:

- Não ser sensível ao caso, podendo aceitar instruções/diretivas/rótulos em maiusculas e minúsculas.
- Desconsiderar tabulações, quebras de linhas e espaços desnecessários em qualquer lugar do código.
- A diretiva CONST deve aceitar números positivos e negativos (somente em decimal). Capacidade de aceitar comentários indicados pelo símbolo “;” em qualquer lugar do código
- O comando COPY deve utilizar um espaço sem vírgula entre os operandos (COPY A B)
- O rótulo de array não tem vírgula entre e a constante mas tem espaço (ADD X + 2).
- Poder criar um rótulo, dar quebra de linha e continuar a linha depois (o rótulo seria equivalente a linha seguinte)

O programa de tradução deve ser capaz de realizar as fases de análise e síntese. O programa deve chamar “**montador**” e receber o arquivo de entrada (com extensão) por argumento na linha de comando (ex: ./montador myprogram.asm). O programa deve dar como saída o arquivo com o mesmo nome MUDANDO a extensão para .OBJ. É obrigatório que o funcionamento do programa seja tal como descrito, caso contrário o trabalho recebe automaticamente a nota ZERO.

O arquivo de saída deve estar em FORMATO TEXTO (não em arquivos binário). Sendo que deve ser uma única linha com espaço entre OPCODE e operandos. No caso da diretiva SPACE deve ser colocado o valor 00 ou 0 (dois ou um dígitos zero) no arquivo objeto (não colocar XX). Lembrando que no arquivo objeto não existe separação entre seções. E sempre no arquivo objeto as constantes e espaços criados por CONST ou SPACE vem depois do programa. Tudo é uma única linha separada por espaços (sem indicar endereços como na sala de aula).

O montador deve identificar os seguintes erros durante a montagem. Montado sempre o programa inteiro e mostrando na tela a(s) LINHA(S) do arquivo original e TIPO DOS ERROS (LEXICO, SINTATICO ou SEMANTICO):

1. declarações e rótulos ausentes;
2. declarações e rótulos repetidos;
3. diretivas e instruções inválidas;
4. seção de texto ausente;
5. erros léxicos,;
6. instruções e diretivas com a quantidade de operando errada;

No Moodle tem arquivos exemplos a serem utilizados. Na correção serão utilizados vários programas além dos disponibilizados no Moodle. Nota: não serão modificados os arquivos de teste para adaptar ao programa desenvolvido pelo grupo. Por exemplo, foi especificado que o COPY deve receber argumentos SEM espaço entre os operandos, o arquivo NÃO será modificado para incluir o espaço para poder rodar.

3.1 EQU e IF

O montador deve aceitar EQU e IF como visto em sala de aula. EQU deve ser sempre declarado antes de qualquer seção. IF somente dentro da seção de texto.

3.2 Ligador

O ligador deve funcionar .\ligador prog1.asm prog2.asm prog3.asm (no maximo 3 programas). O ligador deve internamente chamar o montador para criar os arquivos objetos separadamente para cada arquivo ASM. Os arquivos ASM incluem as diretivas BEGIN, END, PUBLIC e EXTERN. O formato do arquivos objetos saída do montador chamado pelo ligador é livre, já que o seu próprio ligador deve também criar o arquivo .LIG que é o executavél no mesmo formato de ser uma única linha com espaço entre OPCODE e operandos. No caso da diretiva SPACE deve ser colocado o valor 00 ou 0 (dois ou um dígitos zero) no arquivo objeto (não colocar XX). Lembrando que no arquivo objeto não existe separação entre seções. E sempre no arquivo objeto as constantes e espaços criados por CONST ou SPACE vem depois do programa. Tudo é uma única linha separada por espaços (sem indicar endereços como na sala de aula).